



Security Assessment

Draft Report



Safe 4337 Module

August 2026

Prepared for Safe

Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	3
Assessment Methodology.....	4
Threat and Security Overview.....	5
Assets.....	5
Actors.....	6
Roles.....	6
Assumptions.....	7
Attack Surface.....	8
Authorization and signatures.....	8
EntryPoint, nonce, and bundling.....	8
Execution and prefunding.....	9
Initialization and setup.....	10
Documentation and integration.....	10
Findings Summary.....	11
Severity Matrix.....	11
Detailed Findings.....	12
Medium Severity Issues.....	13
M-01: Authorization changes do not invalidate subsequent UserOperations already validated in the same bundle.....	13
Informational Severity Issues.....	19
I-01: Catch-all signature error handling violates ERC-4337 validation semantics.....	19
I-02: Custom SafeOp signing scheme diverges from ERC-4337 userOpHash semantics.....	21
I-03: README reverses validity timestamps in signature encoding.....	23
Disclaimer.....	25
About Certora.....	25

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Safe 4337 Module	https://github.com/safe-fn/dn/safe-modules/tree/4337/v0.3.0	Audited commit: 5931a27 Fix review: f877239	Solidity

Project Overview

This document describes the security review of **Safe 4337 Module**. The work was undertaken from **August 6th, 2026** to **August 11th, 2026**.

The following contract list is included in our scope:

```
safe-modules/modules/4337/contracts/Safe4337Module.sol
safe-modules/modules/4337/contracts/SafeModuleSetup.sol
```

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

The Safe ERC-4337 Module enables a Safe to operate as an ERC-4337 smart account. Installed as both fallback handler and enabled module, it processes validation and execution requests from the configured EntryPoint.

The module verifies owner signatures through the Safe's threshold logic, returns the operation's validity window, and supplies any missing prefund. The EntryPoint handles nonces, replay protection, gas accounting, and bundling. Once validated, the module executes the signed call or delegatecall through the Safe.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

Scope: the Safe ERC-4337 v0.3.0 contracts at commit [5931a275c9f8638cb8c62b366744f4c9fd72533d](#). The primary production path is [Safe4337Module](#), [SafeModuleSetup](#), Safe v1.4.1+, and EntryPoint v0.7.0.

Verdicts describe the implementation state at the time of this review:

- **Impossible** means an enforced invariant or access restriction prevents the threat;
- **Mitigated** means the threat is possible in principle but sufficiently constrained;
- **By design** means the behavior is intentional and its tradeoff is accepted;
- **Audit finding** means a concrete issue identified during this review and tracked in the current audit report. “Audit finding” does not imply prior awareness, confirmation, or acknowledgment by the project team.

Assets

- **Safe-controlled assets and authority** - [executeUserOp](#) can make arbitrary calls or delegatecalls with value through [execTransactionFromModule](#).
- **Owner configuration** - owners, threshold, approved hashes, and ERC-1271 owner behavior determine which [SafeOp](#) values are authorized.
- **Signed UserOperations** - signatures commit to the Safe, nonce, initialization data, execution calldata, gas fields, paymaster data, validity window, EntryPoint, module, and chain.
- **Replay and validity state** - EntryPoint nonce lanes and the returned [validAfter/validUntil](#) window determine when an authorization can execute.
- **EntryPoint deposit and Safe native balance** - [validateUserOp](#) may transfer [missingAccountFunds](#) from the Safe to EntryPoint.
- **Safe configuration and identity** - singleton, proxy factory, initializer, enabled modules, and fallback handler define the counterfactual account and its effective code.

Actors

- **Safe owner – semi-trusted:** signs operations, approves hashes, or supplies contract signatures; an individual owner is bounded by the threshold.
- **Owner quorum – trusted:** can authorize arbitrary Safe calls, delegatecalls, owner changes, module changes, and asset transfers.
- **Bundler/block builder – untrusted:** observes, copies, orders, censors, and submits UserOperations and chooses the fee beneficiary.
- **Paymaster – semi-trusted:** controls sponsorship and post-operation behavior but does not authorize Safe execution.
- **Execution target – untrusted for calls, trusted for delegatecalls:** receives an owner-authorized call; a delegatecall target executes with effective full Safe authority in the Safe's storage context.
- **Supported EntryPoint code – trusted:** validates nonces and time ranges, computes prefund, calls validation, and binds validated operations to execution.
- **Safe proxy – system-only:** stores assets and authorization state, forwards fallback calls, and enforces enabled-module access through singleton logic.
- **Safe singleton and proxy factory – system-only:** provide the account logic and deterministic, atomic deployment path.
- **Safe4337Module – system-only:** adapts EntryPoint callbacks into Safe signature checks, prefunding, and module execution.
- **Module deployer and release tooling – trusted at deployment:** determine the deployed module bytecode and immutable EntryPoint address.
- **Wallet or account integrator – semi-trusted:** selects the singleton, proxy factory, setup helper, module, initializer, validity bounds, and operation data presented for signature.

Roles

- **Fully trusted:** the Safe owner quorum can authorize arbitrary Safe calls, delegatecalls, owner changes, module changes, and asset transfers.
- **System-trusted:** code at `SUPPORTED_ENTRYPOINT` is authorization-critical. Canonical EntryPoint v0.7 binds validation to execution, but malicious or incorrect code at that address can reach module execution without an execution-time signature check.
- **Operation-trusted:** a target selected for delegatecall has arbitrary Safe authority for that execution.

- **Deployment-trusted:** the module deployer determines its bytecode and immutable EntryPoint; the wallet or account integrator determines initial Safe routing. `Safe4337Module` has no owner, upgrade function, or post-deployment admin role.
- **Semi-trusted:** individual owners, ERC-1271 owner contracts, and paymasters can invalidate or delay their own authorization path but cannot independently execute unless the configured threshold permits it.
- **Untrusted:** bundlers, block builders, relayers, calldata submitters, and call targets have no account authorization role.

Assumptions

- Official deployments use the documented v0.3.0 bytecode, Safe v1.4.1 or newer, and canonical EntryPoint v0.7.0 at `0x0000000071727De22E5E9d8BAf0edAc6f37da032`.
- The immutable `SUPPORTED_ENTRYPOINT`, Safe singleton, proxy factory, and deterministic deployment parameters are correct for each chain.
- The 4337 module is both the Safe fallback handler and an enabled module. Additional modules, handlers, and ERC-1271 owners are trusted according to their authority; Safe transaction-guard policy is not assumed to cover module execution.
- Fewer owners than the configured threshold are compromised, and ERC-1271 owner contracts implement the expected semantics.
- EntryPoint correctly binds validation to execution, enforces nonce lanes and validity windows, calculates prefund, and accounts for deposits.
- Wallet software presents the complete signed operation accurately and encodes the validity prefix in the order expected by the contract.
- Counterfactual Safes are deployed and initialized atomically through the intended factory and signed `initCode`.
- Bundler and paymaster simulation, reputation, and availability are operational dependencies for inclusion and liveness, not authorization boundaries.
- Solidity 0.8.23 with the configured optimizer produces bytecode consistent with the reviewed sources; the prior audit explicitly acknowledged optimizer risk.

Attack Surface

Authorization and signatures

- **Threshold bypass during isolated validation** — **Verdict:** Mitigated — `validateUserOp` calls `Safe.checkSignatures`, which enforces the owner set and threshold that exist when validation runs; that authorization can become stale before execution, as covered below.
- **Mutation of signed execution or fee fields** — **Verdict:** Mitigated — `_getSafeOp` commits to every non-signature v0.7 packed `UserOperation` field, the validity window, `Safe`, supported `EntryPoint`, module domain, and chain.
- **Cross-Safe, cross-chain, or cross-module replay** — **Verdict:** Mitigated — the signed structure includes `userOp.sender` and `EntryPoint`, while `domainSeparator` includes chain ID and the module address; `EntryPoint` separately consumes the nonce.
- **Signatures do not authenticate the canonical `userOpHash`** — **Verdict:** Audit finding — identified during this review. `validateUserOp` ignores the `EntryPoint`-supplied hash and verifies a custom `SafeOp` digest. This is not a v0.7 field-integrity gap because `_getSafeOp` covers every non-signature field and adds signed validity bounds, but it deviates from the ERC-4337 interface requirement and duplicates the hashing schema.
- **Unexpected signature-validation errors reported as mismatches** — **Verdict:** Audit finding — identified during this review. `_validateSignatures` catches every `Safe.checkSignatures` revert and returns signature failure, including malformed contract-signature data and unexpected ERC-1271 failures.
- **Compromised threshold authorizes arbitrary execution** — **Verdict:** By design — a quorum can sign arbitrary call, value, and operation parameters and can also change `Safe` owners, modules, and fallback handling.

EntryPoint, nonce, and bundling

- **A non-EntryPoint caller uses the shared module against another Safe** — **Verdict:** Mitigated — through a `Safe` fallback, the appended caller must equal `SUPPORTED_ENTRYPOINT` and `msg.sender` must equal `userOp.sender`. A direct module caller can forge the trailing `_msgSender()` bytes, but validation remains confined to that caller and `executeUserOp*` always operates on `ISafe(msg.sender)`, so it cannot select a third-party `Safe`.
- **A malicious or incorrectly configured EntryPoint bypasses account authorization** — **Verdict:** By design — the module authenticates only the immutable caller address and

does not store or recheck validation in `executeUserOp`. Code configured as `SUPPORTED_ENTRYPOINT` can call the Safe fallback and exercise the enabled module without owner signatures; official deployments assume canonical EntryPoint v0.7 code at that address.

- **Replay of an executed UserOperation — Verdict:** Mitigated — EntryPoint v0.7 maintains a sequence per `(sender, nonce key)` and rejects a consumed sequence; parallel nonce keys are intentional.
- **Owner or threshold changes fail to invalidate later operations in the same bundle — Verdict:** Audit finding — identified during this review. `handleOps` validates every operation and advances each `(sender, nonce key)` sequence before executing any operation; `executeUserOp` does not recheck signatures after an earlier operation changes Safe authorization. A caller can submit the array directly to permissionless `handleOps`; off-chain one-operation-per-unstaked-sender policy is not an on-chain invariant.
- **Bundler changes operation semantics — Verdict:** Mitigated — calldata, gas limits, fees, paymaster data, and initialization data are signed; the unsigned beneficiary only receives EntryPoint-collected fees.
- **Bundler censorship, copying, and ordering — Verdict:** By design — UserOperations are public bearer objects and `handleOps` is permissionless; nonce consumption prevents replay but not censorship, front-running, or adversarial bundle ordering.
- **Signature aggregation is unavailable — Verdict:** By design — `_packValidationData` returns only authorizer `0` or failure sentinel `1`; no aggregator address or aggregated-operation path is implemented by the module.

Execution and prefunding

- **Arbitrary target call or delegatecall — Verdict:** By design — the signed `executeUserOp` parameters are passed to the enabled Safe module, which has unrestricted module authority.
- **Calling unrelated Safe fallback functions through a UserOperation — Verdict:** Mitigated — `validateUserOp` accepts only `executeUserOp` and `executeUserOpWithErrorString` selectors; any nested arbitrary action remains part of the signed wrapper parameters.
- **Failed target execution consumes gas — Verdict:** By design — `executeUserOp` reverts generically and `executeUserOpWithErrorString` bubbles target data; EntryPoint handles failure accounting and charges the operation.

- **Prefund value drains the Safe — Verdict:** Mitigated — under the trusted EntryPoint, `missingAccountFunds` is derived from signed gas parameters and deposit state, the transfer is sent only to `SUPPORTED_ENTRYPOINT`, and EntryPoint verifies the resulting prefund.
- **UserOperations bypass Safe transaction-guard policies — Verdict:** By design — `executeUserOp*` uses `execTransactionFromModule*`; Safe v1.4.1 transaction guards are not invoked, so policies enforced only by `Guard.checkTransaction` do not apply to ERC-4337 execution.

Initialization and setup

- **Counterfactual Safe is initialized by an attacker — Verdict:** Mitigated — the signed `SafeOp` commits to `initCode`; the intended factory atomically creates and initializes the proxy, while Safe `setup` is single-use.
- **SafeModuleSetup enables an unsigned module — Verdict:** Mitigated — the helper has no access control, but a direct call cannot mutate a Safe. It affects a Safe only when delegatecalled in the Safe context; setup commits its calldata in signed `initCode`, while any post-deployment actor able to force that delegatecall already has unrestricted Safe authority.
- **Malicious setup delegatecall — Verdict:** By design — Safe initialization permits arbitrary `to` and `data`; security depends on owners and wallet software approving the exact signed initializer.
- **Module removal or fallback replacement disables ERC-4337 — Verdict:** By design — the owner quorum can change modules and fallback handling; removing either required integration component makes the 4337 path fail.

Documentation and integration

- **Validity timestamps are encoded in the wrong order — Verdict:** Audit finding — identified during this review. `_getSafeOp` parses `validAfter || validUntil`, while the README example encodes and names `validUntil || validAfter`; integrations following the example can invert the validity window.
- **Unsupported EntryPoint or Safe version is integrated — Verdict:** By design — the module enforces only its immutable EntryPoint address and performs no Safe-version or Safe-code-hash check; compatibility outside the documented and audited implementations is an integration assumption.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	–	–	–
Medium	1	1	0
Low	–	–	–
Informational	3	3	1
Total	4	4	1

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
M-01	Authorization changes do not invalidate subsequent UserOperations already validated in the same bundle	Medium	Acknowledged
I-01	Catch-all signature error handling violates ERC-4337 validation semantics	Informational	Acknowledged
I-02	Custom SafeOp signing scheme diverges from ERC-4337 userOpHash semantics	Informational	Acknowledged
I-03	README reverses validity timestamps in signature encoding	Informational	Fixed

Medium Severity Issues

M-01: Authorization changes do not invalidate subsequent UserOperations already validated in the same bundle

Severity: Medium	Impact: High	Likelihood: Low
Files: modules/4337/contracts/ Safe4337Module.sol	Status: Acknowledged	

Description:

The `EntryPoint` validates every `UserOperation` in a bundle before executing any of them. As a consequence, multiple sequential `UserOperations` from the same Safe are all validated against the Safe's owner set and threshold as they existed before execution began.

```
function handleOps(  
    PackedUserOperation[] calldata ops,  
    address payable beneficiary  
) public nonReentrant {  
    // ...  
    unchecked {  
        for (uint256 i = 0; i < opslen; i++) {  
            UserOpInfo memory opInfo = opInfos[i];  
            (  
                uint256 validationData,  
                uint256 pmValidationData  
            ) = _validatePrepayment(i, ops[i], opInfo);  
            _validateAccountAndPaymasterValidationData(  
                i,  
                validationData,  
                pmValidationData,  
                address(0)  
            );  
        }  
    }  
}
```

```
uint256 collected = 0;
emit BeforeExecution();

for (uint256 i = 0; i < opslen; i++) {
    collected += _executeUserOp(i, ops[i], opInfos[i]);
}
// ...
}
```

During validation, `Safe4337Module` calls the Safe's `checkSignatures`, which checks the signatures against the current owners and threshold:

```
// ...
try ISafe(payable(userOp.sender)).checkSignatures(keccak256(operationData),
operationData, signatures) {
    // The timestamps are validated by the entry point, therefore we will not
    check them again
    validationData = _packValidationData(false, validUntil, validAfter);
} catch {
    validationData = _packValidationData(true, validUntil, validAfter);
}
// ...
```

The execution functions, `executeUserOp` and `executeUserOpWithErrorString`, do not verify that this authorization configuration remains unchanged between validation and execution. Consequently, a bundle could contain one or more operations that no longer satisfy the Safe's authorization requirements at execution time because a prior operation changed the owners, the threshold, or both.

For example:

1. A Safe is configured with owners `A`, `B`, and `C` under a threshold of 1.
2. `A` and `B` suspect that `C`'s signing key has been compromised, so they submit a `UserOperation` that removes `C` from the owner set.

3. The attacker controlling **C** observes the operation in the shared **UserOperation** mempool and signs another **UserOperation** that transfers the Safe's assets to an attacker-controlled address.
4. The attacker copies the first operation and submits it alongside the attacker's operation in the same call to the permissionless **handleOps** function, either directly or through a cooperating bundler.
5. **EntryPoint** validates both operations while the owners are still **A**, **B**, and **C** and the threshold is still 1, even though the first operation will remove **C** from the owner set.
6. Both operations execute successfully. The Safe ends up with only **A** and **B** as owners, but its assets have already been transferred to the attacker.

Another scenario could be one in which an attacker may possess a previously signed operation at sequence **s + 1** without possessing sufficient signatures to create an operation at the currently required sequence **s**:

1. A Safe is configured with owners **A**, **B**, and **C** under a threshold of 2. For nonce key **K**, the next expected sequence number is **s**.
2. Before **C** is compromised, a withdrawal transferring assets to **C**'s wallet is created at sequence **s + 1** and signed by **A** and **C**. The attacker later obtains this complete, signed operation after compromising **C**.
3. The attacker cannot execute the withdrawal independently because **EntryPoint** still expects sequence **s**. The attacker cannot change its sequence number because the nonce is covered by the signatures, and **C** alone cannot create a valid operation at sequence **s** under the 2-of-3 threshold.
4. After discovering the compromise, **A** and **B** publish an operation at sequence **s** that removes **C** while preserving the threshold of 2.
5. The attacker copies the signed removal operation and submits it together with the old withdrawal in the same call to the permissionless **handleOps** function, ordering the removal at sequence **s** before the withdrawal at sequence **s + 1**.
6. **EntryPoint** validates both operations before executing either. The removal is validly authorized by **A** and **B**, and the withdrawal is validly authorized by **A** and **C** because **C** is still an owner during validation. Their consecutive sequences also satisfy the nonce requirements.
7. **EntryPoint** then executes the removal, making **A** and **B** the only owners. Nevertheless, the withdrawal executes afterward because **executeUserOp** does not recheck its signatures against the updated owner set.

8. The assets are transferred to the compromised wallet even though `C` had already been removed. If the withdrawal's signatures had been checked during execution, they would have failed because `C` was no longer an owner.

Instead of immediately transferring assets, the stale operation could add an attacker-controlled owner with threshold 1, execute a malicious delegatecall, or exercise external protocol-administration privileges, resulting in persistent control over the Safe and any systems it administers.

Removing an owner or increasing the Safe threshold does not necessarily revoke operations authorized under the previous configuration. An operation carrying stale authorization can execute after the revocation if it appears later in the same `EntryPoint` bundle and was validated before the revocation took effect.

This violates the ordering semantics users would normally expect: with ordinary Safe transactions, signatures are checked during each transaction's execution, so a transaction signed only by a removed owner cannot execute after the owner-removal transaction.

Exploitation requires signatures that are valid under the previous Safe policy. A compromised owner who is still authorized could often attempt to execute or front-run a malicious transaction before being removed, reducing the incremental impact. Nevertheless, this behavior remains relevant when revocation operations are submitted privately to avoid a public-mempool race, when previously signed or queued operations remain outstanding, or when workflows rely on nonce ordering to make an authorization change effective before subsequent operations.

Recommendations:

The best remediation would be to recheck the signatures during the execution of every operation, before calling either `execTransactionFromModule` or `execTransactionFromModuleReturnData`.

For configurations in which owner authorization is fully determined by the Safe's direct owner set and threshold, an alternative would be to enforce the following invariant for every `UserOperation`:

The Safe's owner set and threshold immediately before execution must match the owner set and threshold against which the `UserOperation` was validated.

Bind every `UserOperation` to the Safe's authorization configuration under which it was validated and verify that configuration again immediately before executing its requested call.

One approach is to change the existing execution ABI to accept an `expectedAuthorizationState` value:

```
function executeUserOp(
    address to,
    uint256 value,
    bytes calldata data,
    uint8 operation,
    bytes32 expectedAuthorizationState
) external onlySupportedEntryPoint {
    if (_authorizationState(msg.sender) != expectedAuthorizationState) {
        revert AuthorizationStateChanged();
    }

    if (!ISafe(msg.sender).execTransactionFromModule(to, value, data, operation))
    {
        revert ExecutionFailed();
    }
}

function _authorizationState(address safe) internal view returns (bytes32) {
    return keccak256(
        abi.encode(
            ISafe(safe).getOwners(),
            ISafe(safe).getThreshold()
        )
    );
}
```

The client would calculate this commitment from the Safe's current owners and threshold and include it in `userOp.callData`. Because the operation hash signed by the owners covers `userOp.callData`, the owner signatures would automatically cover the commitment.

`validateUserOp` must decode the commitment from the calldata and verify that it equals the Safe's authorization state during validation. `executeUserOp` must then repeat the comparison

during execution. Both checks are required: without the validation-time comparison, the signers could provide a commitment to an anticipated future owner configuration while having the operation validated under the old configuration.

The execution-time comparison occurs before the requested call. Therefore, a **UserOperation** authorized under the current policy may itself change the Safe's owners or threshold, but any later operation in the bundle that was validated under the previous policy will revert before performing its requested action.

This ensures that the Safe's direct owner set and threshold have not changed between validation and execution. However, it does not detect changes to authorization state internal or external to an EIP-1271 contract owner. Supporting such owners requires execution-time signature revalidation or an equivalent commitment to their complete authorization state.

Customer Response:

Acknowledged, will consider a change in the future iteration.

Informational Severity Issues

I-01: Catch-all signature error handling violates ERC-4337 validation semantics

Description:

`_validateSignatures()` reports every error from `Safe.checkSignatures()` as an ordinary signature mismatch. This does not follow ERC-4337's validation requirements.

ERC-4337 recommends returning `SIG_VALIDATION_FAILED` (1) when a well-formed signature does not match. Other errors must revert. This distinction allows bundlers to differentiate an invalid signature from malformed input or broken account validation.

However, the module catches every revert:

```
try ISafe(payable(userOp.sender)).checkSignatures(
    keccak256(operationData),
    operationData,
    signatures
) {
    validationData = _packValidationData(false, validUntil, validAfter);
} catch {
    validationData = _packValidationData(true, validUntil, validAfter);
}
```

`checkSignatures()` may revert because the signature is invalid, but it also reverts when:

- The signature data is too short.
- A contract-signature offset is malformed or out of bounds.
- The Safe has an invalid threshold.
- An ERC-1271 owner unexpectedly reverts.

These are not ordinary signature mismatches. Under ERC-4337, they should propagate as validation errors rather than return `SIG_VALIDATION_FAILED`.

This does not allow an invalid operation to execute. Both outcomes reject the `UserOperation`. The impact is limited to non-compliant validation behavior and inaccurate bundler diagnostics. In practice, it is most likely to affect malformed `UserOperations`, faulty integrations, or contract-based owners that revert unexpectedly.

Recommendations:

Return `SIG_VALIDATION_FAILED` only for genuine signature mismatches. Validate the Safe signature encoding before calling `checkSignatures()` and let malformed input revert. Unexpected ERC-1271 validator errors should also propagate.

If necessary, introduce a validation adapter that distinguishes known Safe signature failures from structural or unexpected errors instead of using a catch-all block.

Customer Response:

Acknowledged

I-02: Custom SafeOp signing scheme diverges from ERC-4337 userOpHash semantics

Description:

`validateUserOp()` ignores the `userOpHash` supplied by the EntryPoint. Instead, it reconstructs and validates a separate EIP-712 `SafeOp` hash:

```
function validateUserOp(
    PackedUserOperation calldata userOp,
    bytes32,
    uint256 missingAccountFunds
) external onlySupportedEntryPoint returns (uint256 validationData) {
    // ...
    validationData = _validateSignatures(userOp);
}
```

[ERC-4337](#) specifies that an account must validate that the operation signature is valid for the EntryPoint-provided `userOpHash`:

MUST validate that the signature is a valid signature of the `userOpHash`, and SHOULD return `SIG_VALIDATION_FAILED` (1) without reverting on signature mismatch. Any other error MUST revert.

This is the canonical operation hash calculated by the EntryPoint and bound to the EntryPoint and chain.

The module instead derives `operationData` independently from the UserOperation fields and validates its hash:

```
(bytes memory operationData, , , bytes calldata signatures) = _getSafeOp(userOp);
try ISafe(payable(userOp.sender)).checkSignatures(
    keccak256(operationData), operationData, signatures
) {
    // ...
}
```

The custom hash covers all security-relevant fields of the scoped `PackedUserOperation` and adds the validity timestamps and module address. No unsigned-field substitution or authorization bypass was identified. However, Safe owner signatures authenticate a different hash from the one used by generic ERC-4337 tooling, events, and receipts.

The implementation also duplicates the EntryPoint's field-coverage logic. A future UserOperation or EntryPoint change must be reflected independently in `SafeOp`, increasing the risk that a newly introduced field or domain component is omitted.

The impact is therefore limited to standards compliance, interoperability, and maintenance risk.

Recommendations:

Use the EntryPoint-provided `userOpHash` as the root of the Safe authorization. If Safe-specific EIP-712 data and validity timestamps are required, sign a wrapper containing `userOpHash`, `validAfter`, and `validUntil`.

```
// Instead of hashing every UserOperation field again:
safeOpHash = hashTypedData(
    SafeOp({userOpHash, validAfter, validUntil})
);
safe.checkSignatures(safeOpHash, safeOpData, signatures);
```

This keeps the Safe-specific domain and validity range while deriving authorization from the canonical operation hash calculated by the EntryPoint.

If the custom `SafeOp` representation is retained, document the deviation and maintain tests proving that every field included in the EntryPoint's `userOpHash` also changes the Safe operation hash.

Customer Response:

Acknowledged

I-03: README reverses validity timestamps in signature encoding

Description:

The README instructs integrators to prefix Safe signatures with `validUntil` followed by `validAfter`:

```
function encodeSignatures(uint48 validUntil, uint48 validAfter, bytes signatures)
{
    return abi.encodePacked(validUntil, validAfter, signatures);
}
```

However, `_getSafeOp()` decodes the first six bytes as `validAfter` and the next six bytes as `validUntil`:

```
validAfter = uint48(bytes6(sig[0:6]));
validUntil = uint48(bytes6(sig[6:12]));
signatures = sig[12:];
```

Because both fields have the same type and width, following the README does not produce a decoding error. Instead, the module silently interprets the validity bounds in reverse.

For a typical finite window where `validAfter < validUntil`, this produces an impossible window and the EntryPoint rejects the UserOperation. More significantly, an expiration-only operation intended to use `validAfter = 0` and `validUntil = T` is interpreted as `validAfter = T` and `validUntil = 0`. ERC-4337 treats a zero `validUntil` as infinite, so an operation intended to expire at `T` may instead become valid at `T` and remain valid indefinitely if the integration signs the digest derived from the incorrectly encoded prefix.

The repository's TypeScript utilities and tests use the order expected by the contract, `validAfter || validUntil`. The issue therefore affects third-party integrations that implement signature encoding from the README rather than using those utilities. Depending on how the operation hash is constructed, the result is either an invalid signature or unintended validity semantics.

Recommendations:

Update the README helper to encode `validAfter` before `validUntil`:

```
function encodeSignatures(uint48 validAfter, uint48 validUntil, bytes signatures)
{
    return abi.encodePacked(validAfter, validUntil, signatures);
}
```

Keep the documented encoding synchronized with the contract and TypeScript utilities, and add a documentation or integration test covering an expiration-only operation where `validAfter = 0` and `validUntil` is non-zero.

Customer Response:

Fixed in [PR#539](#)

Fix Review:

Fix confirmed

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.